

Unit – 1 Node Js

What is Node.js?

- Node.js is an open source server environment
- Node.js is free
- Node.js runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)
- Node.js uses JavaScript on the server

Why Node.js?

- Node.js uses asynchronous programming!
 - A common task for a web server can be to open a file on the server and return the content to the client.
 - Here is how PHP or ASP handles a file request:
 - Sends the task to the computer's file system.
 - Waits while the file system opens and reads the file.
 - Returns the content to the client.
 - Ready to handle the next request.
 - Here is how Node.js handles a file request:
 - Sends the task to the computer's file system.
 - Ready to handle the next request.
 - When the file system has opened and read the file, the server returns the content to the client.
 - Node.js eliminates the waiting, and simply continues with the next request.
 - Node.js runs single-threaded, non-blocking, asynchronous programming, which is very memory efficient.
-

What Can Node.js Do?

- Node.js can generate dynamic page content
- Node.js can create, open, read, write, delete, and close files on the server
- Node.js can collect form data
- Node.js can add, delete, modify data in your database
- ---
- What is a Node.js File?
- Node.js files contain tasks that will be executed on certain events
- A typical event is someone trying to access a port on the server
- Node.js files must be initiated on the server before having any effect
- Node.js files have extension ".js"

/Installing Download Node.js

The official Node.js website has installation instructions for Node.js: <https://nodejs.org>

Once you have downloaded and installed Node.js on your computer, let's try to display "Hello World" in a web browser.

Create a Node.js file named "myfirst.js", and add the following code:

```
const http = require('node:http');
```

```
const hostname = '127.0.0.1';  
const port = 3000;
```

```
const server = http.createServer((req, res) => {  
  res.statusCode = 200;  
  res.setHeader('Content-Type', 'text/plain');  
  res.end('Hello, World!\n');  
});  
  
server.listen(port, hostname, () => {  
  console.log(`Server running at  
http://${hostname}:${port}/`);  
});
```

Save the file on your computer: C:\Users\Your Name\myfirst.js

What is npm

npm is a short form of *Node Package Manager*, which is the world's largest software registry. The open-source web project developers use it from the entire world to share and borrow packages. The npm also acts as a command-line utility for the Node.js project for installing packages in the project, dependency management, and even version management.

Components of npm

npm mainly consists of three different components, these are:

1. **Website:** The npm official [website](#) is used to find packages for your project, create and set up profiles to manage and access private and public packages.
2. **Command Line Interface (CLI):** The CLI runs from your computer's terminal to interact with npm packages and repositories.
3. **Registry:** The registry is a large and public database of JavaScript projects and meta-information. You can use

any supported npm registry that you want or even your own. You can even use someone else's registry as per their terms of use.

When to use npm

- Use npm to adapt packages of code for your apps or incorporate packages as they are.
- Use npm when you need to share code with any npm user, anywhere.
- npm can be used when you want to restrict code to specific developers.
- Use npm to create organizations to coordinate package maintenance, coding, and developers.
- Use npm to manage multiple versions of code and code dependencies.
- Use npm to update applications easily when the underlying code is updated.
- Use npm to find other developers who are working on similar problems and projects.

This means that you have to install Node.js to get npm installed on your computer.

Download Node.js from the official Node.js web site: <https://nodejs.org>

How to Use NPM with Node.js?

To start using NPM in your project, follow these simple steps

Step 1: Install Node.js and NPM

First, you need to install Node.js. NPM is bundled with the Node.js installation. You can follow our article to Install the Node and NPM- [How to install Node on your system](#)

Step 2: Verify the Installation

After installation, verify Node.js and NPM are installed by running the following commands in your terminal:

`node -v`

`npm -v`

These commands will show the installed versions of Node.js and NPM.

```
Your environment has been set up for using Node.js 22.12.0 (x64) and npm.
```

```
C:\Users\User>-v  
'-v' is not recognized as an internal or external command,  
operable program or batch file.
```

```
C:\Users\User>v  
'v' is not recognized as an internal or external command,  
operable program or batch file.
```

```
C:\Users\User> npm -v  
10.9.0
```

```
C:\Users\User>|
```

Step 3: Initialize a New Node.js Project

In the terminal, navigate to your project directory and run:

`npm init -y`

This will create a package.json file, which stores metadata about your project, including dependencies and scripts.

```
Your environment has been set up for using Node.js 22.12.0 (x64)

C:\Users\User>-v
'-v' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\User>v
'v' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\User> npm -v
10.9.0

C:\Users\User>npm init -y
Wrote to C:\Users\User\package.json:

{
  "name": "user",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

Step 4: Install Packages with NPM

To install a package, use the following command

For example, to install the Express.js framework

npm install express

```
C:\Users\User>npm install express

added 68 packages, and audited 69 packages in 7s

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

This will add express to the node_modules folder and automatically update the package.json file with the installed package information.

Using NPM Package in the project

Create a file named `app.js` in the project directory to use the package

```
//app.js
```

```
const express = require('express');//import the required package
```

```
const app = express();
```

```
app.get('/', (req, res) => {  
  res.send('Hello, World!');  
});
```

```
app.listen(3000, () => {  
  console.log('Server running at http://localhost:3000');  
});
```

- `express()` creates an instance of the [Express app](#).
- `app.get()` defines a route handler for HTTP GET requests to the root (`/`) URL.
- `res.send()` sends the response “Hello, World!” to the client.
- `app.listen(3000)` starts the server on port 3000, and `console.log()` outputs the server URL.

Now run the application with

```
node app.js
```

Visit `http://localhost:3000` in your browser, and you should see the message: Hello, World!

Popular NPM Packages

NPM has a massive library of packages. Here are a few popular packages that can enhance your Node.js applications:

- [Express](#): A fast, minimal web framework for building APIs and web applications.
- [Mongoose](#): A MongoDB object modeling tool for Node.js.

- [Lodash](#): A utility library delivering consistency, customization, and performance.
- [Axios](#): A promise-based HTTP client for making HTTP requests.
- [React](#): A popular front-end library used to build user interfaces.

What is Node.js REPL?

REPL stands for:

- **Read**: The REPL reads user input (JavaScript code).
- **Eval**: The input is evaluated (executed) by the [JavaScript engine](#).
- **Print**: The result of the evaluation is printed to the screen.
- **Loop**: The process repeats (loops), waiting for the next input.
- This interactive environment allows developers to test small snippets of code without needing to create a full script. The REPL is built directly into [Node.js](#), so no additional setup is required.
- The Node.js REPL environment allows developers to test code snippets quickly. To build real-world full-stack applications .

Getting Started with REPL

To start working with REPL environment of NODE; open up the terminal (in case of UNIX/LINUX) or the Command prompt (in case of Windows) and write **node** and press 'enter' to start the REPL.

Node

```
C:\Users\User>node
Welcome to Node.js v22.12.0.
Type ".help" for more information.
> |
```

The REPL has started and is demarcated by the '>' symbol. Various operations can be performed on the REPL. Below are

some of the examples to get familiar with the REPL environment.

Key Features of Node.js REPL

1. Executing JavaScript Code

The REPL is a full-featured JavaScript environment, meaning you can run any valid JavaScript code inside it.

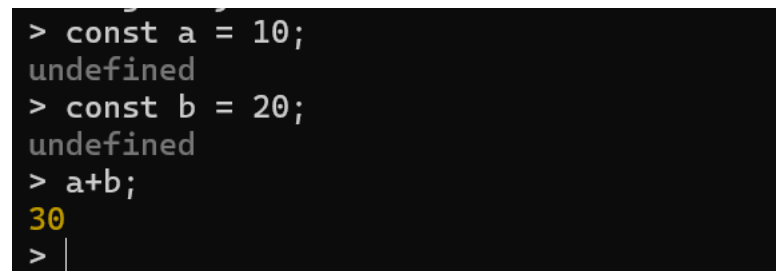
Example:

```
> const x = 10;
```

```
> const y = 20;
```

```
> x + y
```

```
30
```

A screenshot of the Node.js REPL environment. It shows a dark background with light-colored text. The input and output are as follows:

```
> const a = 10;
undefined
> const b = 20;
undefined
> a+b;
30
> |
```

You can declare variables, create functions, and run any code that would work in a regular JavaScript runtime.

2. Multi-Line Input

In case of complex logic (like loops or functions), the REPL supports multi-line input. When you enter a block of code, the REPL will continue reading input until the block is complete.

Example:

```
> function add(a, b) {
```

```
...   return a + b;
```

```
... }
```

```
> add(5, 10)
```

```
15
```

Here, the REPL waits for you to complete the function block before evaluating the code.

3. Underscore (_) Variable

The REPL provides a special variable `_` (underscore) that stores the result of the last evaluated expression.

Example:

```
> 3 + 3
```

```
6
```

```
> _ * 2
```

In this case, the result of `3 + 3` is stored in `_`, which is then used in the next line to calculate 12.

4. Saving and Loading REPL Sessions

The REPL allows you to save the session output to a file and load previously saved sessions, making it easier to keep track of the code you've tested.

Saving a Session: To save your REPL session to a file, use the `.save` command:

```
> .save ./repl_session.js
```

Loading a Session: You can load the saved session into a new REPL session using `.load`:

```
> .load ./repl_session.js
```

5. Accessing Node.js Core Modules

The REPL environment allows you to load and use Node.js core modules, such as `fs`, `path`, `http`, etc., without needing to exit the REPL or write an entire script.

Example:

```
> const fs = require('fs');
```

```
> fs.readFileSync('test.txt', 'utf8');
```

In this example, the `fs` (file system) module is loaded, and the REPL reads the content of a file named `test.txt`.

6. Error Handling in REPL

The REPL is forgiving of errors and will display helpful error messages without crashing the entire session. This makes it a safe environment for testing.

Example:

> console.log(foo);

ReferenceError: foo is not defined

Built-in REPL Commands

Node.js REPL provides several built-in commands (REPL commands always start with a dot .).

- **.help:** Displays a list of all available REPL commands.
- **.break:** Breaks out of multi-line input or clears the current input.
- **.clear:** Resets the REPL context by clearing all declared variables.
- **.exit:** Exits the REPL session.
- **Example:** Performing Arithmetical operations in REPL

```
C:\Users\Parikshit\Desktop\GeeksForGeeks>node
> 10+20
30
> 1-2
-1
> 10/4
2.5
> 10%3
1
>
```

- **Example:** Performing operations using libraries of NODE.
- **MATH** library is being used in below example.

```
C:\Users\Parikshit\Desktop\GeeksForGeeks>node
> Math.ceil(10/3)
4
> Math.floor(10/3)
3
> math.round(math.e, 3);
ReferenceError: math is not defined
> Math.round(Math.e, 3)
NaN
> Math.round(2.57362, 3)
3
> Math.sqrt(16)
4
> Math.pow(3, 4)
81
>
(To exit, press ^C again or type .exit)
>
```

- C:\Users\Parikshit\Desktop\GeeksForGeeks>node

Note: using **'math'** shows error as the library is referenced as **'Math'** in NODE and not **'math'**.

Example: Using variables in REPL. The keyword **var** is used to assign values to variables.

```
C:\Users\Parikshit\Desktop\GeeksForGeeks>node
> var a = 4;
undefined
> a
4
> var c = "GeeksForGeeks"
undefined
> c
'GeeksForGeeks'
```

What is Blocking?

It refers to the blocking of further operation until the current operation finishes. Blocking methods are executed synchronously. Synchronously means that the program is executed line by line. The program waits until the called function or the operation returns.

Example: Following example uses the [readFileSync\(\)](#) function to read files and demonstrate Blocking in Node.js

```
const fs = require('fs');
```

```
const filepath = 'text.txt';
```

```
// Reads a file in a synchronous and blocking way
const data = fs.readFileSync(filepath, {encoding: 'utf8'});
```

```
// Prints the content of file
console.log(data);
```

```
// This section calculates the sum of numbers from 1 to 10
let sum = 0;
for(let i=1; i<=10; i++){
    sum = sum + i;
```

```
}
```

```
// Prints the sum  
console.log('Sum: ', sum);
```

Run the **index.js** file using the following command:

```
node index.js
```

Output:

This is from text file.

Sum: 55

What is Non-Blocking ?

It refers to the program that does not block the execution of further operations. Non-Blocking methods are executed asynchronously. Asynchronously means that the program may not necessarily execute line by line. The program calls the function and move to the next operation and does not wait for it to return.

Example: Following example uses the [readFile\(\)](#) function to read files and demonstrate Non-Blocking in Node

```
const fs = require('fs');
```

```
const filepath = 'text.txt';
```

```
// Reads a file in a asynchronous and non-blocking way
```

```
fs.readFile(filepath, {encoding: 'utf8'}, (err, data) => {
```

```
    // Prints the content of file
```

```
    console.log(data);
```

```
});
```

```
// This section calculates the sum of numbers from 1 to 10
```

```
let sum = 0;
```

```
for(let i=1; i<=10; i++){
```

```
    sum = sum + i;
```

```
}
```

```
// Prints the sum  
console.log('Sum: ', sum);
```

Run the **index.js** file using the following command:

```
node index.js
```

Output:

Sum: 55

This is from text file.

Node.js Callbacks

Callback is an asynchronous equivalent for a function. It is called at the completion of each task. In Node.js, callbacks are generally used. All APIs of Node are written in a way to support callbacks. **For example:** when a function starts reading a file, it returns the control to the execution environment immediately so that the next instruction can be executed.

In Node.js, once file I/O is complete, it will call the callback function. So there is no blocking or wait for File I/O. This makes Node.js highly scalable, as it can process a high number of requests without waiting for any function to return a result.

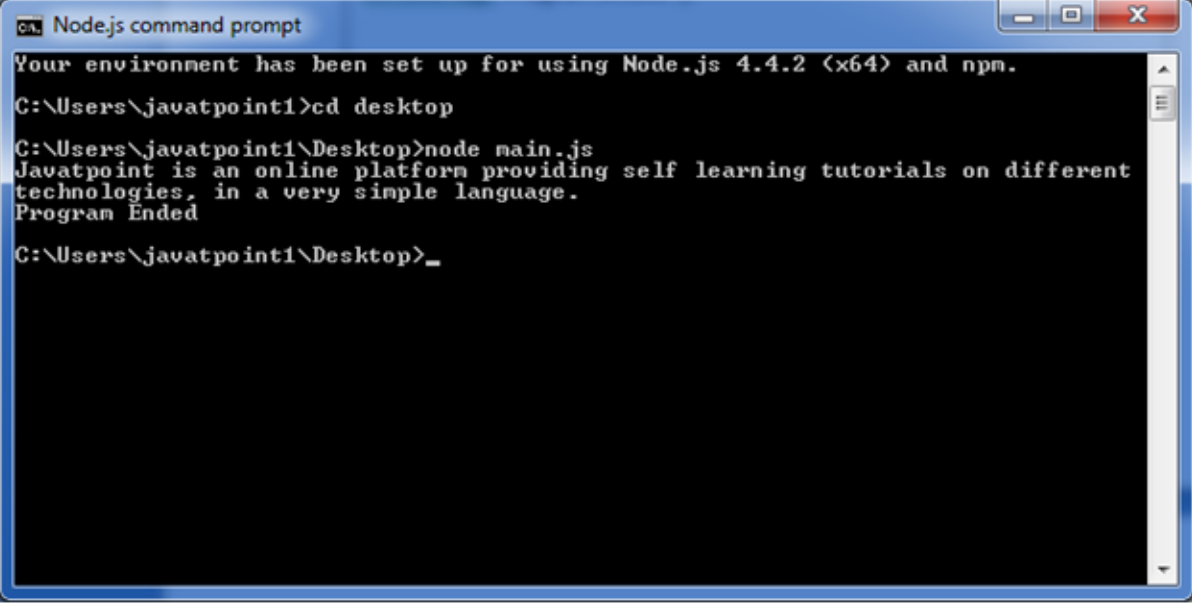
Blocking Code Example

Follow these steps:

1. Create a text file named **input.txt** having the following content:
 1. **Parul University** is NAAC A++ accredited private university in Vadodara, Gujarat. Explore our diverse educational programs
 2. different technologies, in a very simple language.
2. Create a JavaScript file named **main.js** having the following code:
 1. `var fs = require("fs");`
 2. `var data = fs.readFileSync('input.txt');`
 3. `console.log(data.toString());`
 4. `console.log("Program Ended");`

3. Open the **Node.js** command prompt and execute the following code.

1. node main.js



```
Node.js command prompt
Your environment has been set up for using Node.js 4.4.2 <x64> and npm.
C:\Users\javatpoint1>cd desktop
C:\Users\javatpoint1\Desktop>node main.js
Javatpoint is an online platform providing self learning tutorials on different
technologies, in a very simple language.
Program Ended
C:\Users\javatpoint1\Desktop>_
```

Non Blocking Code Example

Follow these steps:

Create a text file named input.txt having the following content:

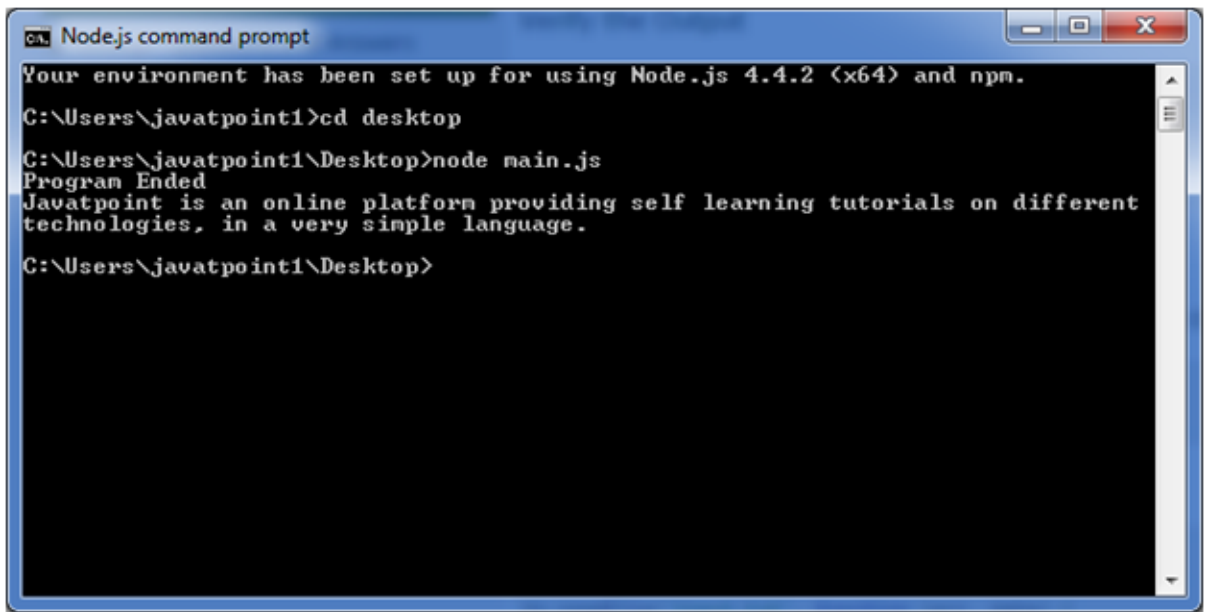
1. **Parul University** is NAAC A++ accredited private university in Vadodara, Gujarat. Explore our diverse educational programs
2. different technologies, in a very simple language.

Create a JavaScript file named main.js having the following code:

1. var fs = require("fs");
2. fs.readFile('input.txt', function (err, data) {
3. if (err) return console.error(err);
4. console.log(data.toString());
5. });
6. console.log("Program Ended");

Open the Node.js command prompt and execute the following code.

1. node main.js



```
Node.js command prompt
Your environment has been set up for using Node.js 4.4.2 (x64) and npm.
C:\Users\javatpoint1>cd desktop
C:\Users\javatpoint1\Desktop>node main.js
Program Ended
Javatpoint is an online platform providing self learning tutorials on different
technologies, in a very simple language.
C:\Users\javatpoint1\Desktop>
```

Node.js Web Server

- Node.js is a powerful JavaScript runtime that enables developers to build scalable and efficient web servers.
- In this article, we'll walk through the steps to create a simple web server using Node.js, covering the basics and providing a foundation for more advanced applications.
- [Node.js](#) has a built-in module called HTTP, which allows Node.js to transfer data over the Hypertext Transfer Protocol (HTTP).
- The HTTP module can create an HTTP server that listens to server ports and gives a response back to the client.

Example 1: Implementation to show the web server creation in Node.js.

```
// Import the Node.js http module
const http = require('http');
```

```
// req is the request object which is
// coming from the client side
// res is the response object which is going
// to client as response from the server

// Create a server object
http.createServer(function (req, res) {

    // 200 is the status code which means
    // All OK and the second argument is
    // the object of response header.
    res.writeHead(200, { 'Content-Type': 'text/html' });

    // Write a response to the client
    res.write('Congrats you have a created a web server');

    // End the response
    res.end();

}).listen(8081); // Server object listens on port 8081

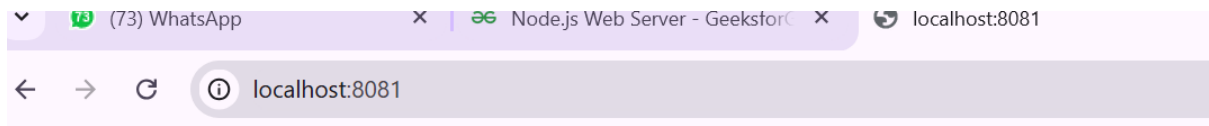
console.log('Node.js web server at port 8081 is running..')
```

```
PS C:\Users\User\Documents\Nodejs> node webserver.js
Node.js web server at port 8081 is running..
█
```



localhost:8081|





Congrats you have a created a web server

Global Objects in Node.js

Node.js Global Objects are the objects that are available in all modules. These are built-in objects that are part of the JavaScript and can be used directly in the application without importing any particular module.

The Node.js Global Objects are listed below:

1. Class: Buffer

The Buffer class is an inbuilt globally accessible class that means it can be used without importing any module. The Buffer class is used to deal with binary data. Buffer class objects are used to represent binary data as a sequence of bytes.

Command Prompt - node

```
Microsoft Windows [Version 10.0.18362.836]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\User>node
Welcome to Node.js v12.15.0.
Type ".help" for more information.
> const buffer = new Buffer.alloc(5,"abcde");
undefined
> console.log(buffer)
<Buffer 61 62 63 64 65>
undefined
> _
```

- **console:** It is an inbuilt global object used to print to stdout and stderr.

```
> console.log("Hello World!!!");
Hello World!!!
undefined
> console.error("An error occurred");
An error occurred
undefined
> _
```

- **process:** It is an inbuilt global object that is an instance of EventEmitter used to get information on current process. It can also be accessed using require() explicitly.

```
C:\Users\User>node desktop/gfg/demo.js
Within Application.
Application Before Exit: 0
Application Exiting: 0
```

2. global:

It is a global namespace. Defining a variable within this namespace makes it globally accessible.

`var myvar`

It is a global scope when declared within the browser. However, any variable defined within a node.js file is accessible only within that file.

- **setImmediate() method:** It schedules the immediate execution of the callback. The callback functions are queued and executed in the order in which they are created. The callback queue is processed at every event loop iteration. If there is a timer queued inside the executing callback, the timer will not get triggered until the next event loop iteration.
- **clearImmediate() method:** It stops the immediate object returned by the `setImmediate()` method.
- **setInterval() method:** It executes the callback function at repeated intervals. If an interval is larger than 2147483647 or less than 1, the interval is set to 1. Non-integer delays are truncated to the nearest integer.
- **clearInterval() method:** It stops the interval object created by `setInterval()` method.
- **setTimeout() method:** It is a global function used to run a callback function after at least delay in milliseconds. Node.js does not guarantee the exact timing of when callbacks will fire but tries to maintain the timing as close as possible to the specified delay. Any delay larger than 2147483647 or less than 1, is set to 1 automatically. Non-integer delays are truncated to the nearest integer.

```
function printHello() {
  console.log( "Hello, World!");
}
```

```
// Now call above function after 2 seconds
var timeoutObj = setTimeout(printHello, 2000);
```

```
C:\Users\User>node desktop/gfg/demo.js
Hello, World!
```

- **clearTimeout() method:** The clearTimeout() method is used to cancel or stop a timeout that was set with setTimeout() method. The timeoutObj is the object returned by setTimeout() method.
- **queueMicrotask() method:** A microtask is a short function that is executed after the callback function exits and only if the JavaScript execution stack is empty. The queueMicrotask() method is used to execute such functions after the callback function completes successfully. If the callback function does not return the control to other JavaScript code, the event loop runs all of the microtasks in the microtask queue. The microtask queue is processed multiple times per iteration of the event loop. If a microtask adds more microtasks to the queue then the newly-added microtasks execute before the next task is run. This is because the event loop keeps calling microtasks until there are none left in the queue.

3. TextEncoder:

It is an implementation of the WHATWG Encoding Standard TextEncoder API. All instances of TextEncoder are encoded in UTF-8 only.

```
C:\Users\User>node
Welcome to Node.js v12.15.0.
Type ".help" for more information.
> const encoder = new TextEncoder();
undefined
> const e = encoder.encode("Welcome to GFG");
undefined
> console.log(e);
Uint8Array [
  87, 101, 108, 99, 111,
  109, 101, 32, 116, 111,
  32, 71, 70, 71
]
undefined
```

4. TextDecoder:

It is an implementation of the WHATWG Encoding Standard TextDecoder API.

```
C:\Users\User>node
Welcome to Node.js v12.15.0.
Type ".help" for more information.
> const encoder = new TextEncoder();
undefined
> const e = encoder.encode("Welcome to GFG");
undefined
> console.log(e);
Uint8Array [
  87, 101, 108, 99, 111,
  109, 101, 32, 116, 111,
  32, 71, 70, 71
]
undefined
> const decoder = new TextDecoder();
undefined
> const d = decoder.decode(e);
undefined
> console.log(d)
Welcome to GFG
undefined
>
```

5. Class: URL

The URL class instance is a global object and is implemented by the following WHATWG URL Standard. The URL constructor creates a new URL object as shown below. /foo is the input and <https://www.helloworld.org/> is the base value.

```
> const url = new URL("/foo","https://www.helloworld.org/");
undefined
> console.log(url)
URL {
  href: 'https://www.helloworld.org/foo',
  origin: 'https://www.helloworld.org',
  protocol: 'https:',
  username: '',
  password: '',
  host: 'www.helloworld.org',
  hostname: 'www.helloworld.org',
  port: '',
  pathname: '/foo',
  search: '',
  searchParams: URLSearchParams {},
  hash: ''
}
undefined
>
```

6. URLSearchParams:

URLSearchParams API is used to perform read and write operations on the query of a URL.

```
const myURL = new
URL('https://www.register.com/?name=gfg');

// It prints gfg
console.log(myURL.searchParams.get('name'));

myURL.searchParams.append('name', 'xyz');

// It prints https://www.register.com/?name=gfg&name=xyz
console.log(myURL.href);
```

7. WebAssembly:

The global object that acts as a namespace for all W3C WebAssembly related functionality. WebAssembly is a low level Assembly-like language that can be run on modern browsers. The following variables might appear to be global but actually exist only within the scope of some modules.

- **require(id) method:** It is used to import modules and returns an object of 'any' datatype.

```
const express = require('express')
```

- **exports:** It is used to exports modules using `module.exports`.
- **module:** It is a reference to the current module and is not global rather local to each module. It is used to make a particular module available through `require()` in the application.
- **__dirname:** The output throws an error which proves that `__dirname` is not globally defined in node.js. It requires a script to give the desired output as `__dirname` is only defined in scripts.

 Command Prompt - node

```
undefined
> console.log(__dirname)
Thrown:
ReferenceError: __dirname is not defined
> _
```

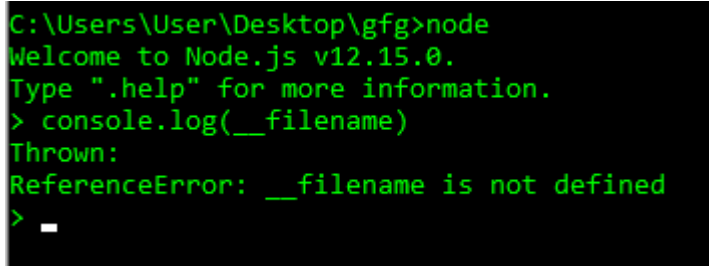
- Create a demo.js file
- Paste the following code:

```
console.log("__dirname : "+ __dirname);
```

- Run the demo.js file

8. __filename:

The output throws an error which proves that __filename is not globally defined in node.js. It requires a script to give the desired output as __filename is only defined in scripts.

A screenshot of a terminal window with a black background and green text. The text shows a user running 'node' in a directory 'C:\Users\User\Desktop\gfg'. The terminal displays the Node.js welcome message and the user's command 'console.log(__filename)'. It then shows a 'ReferenceError: __filename is not defined' message.

```
C:\Users\User\Desktop\gfg>node
Welcome to Node.js v12.15.0.
Type ".help" for more information.
> console.log(__filename)
Thrown:
ReferenceError: __filename is not defined
> _
```

- Create a demo.js file
- Paste the following code:

```
console.log("__filename : "+ __filename);
```

- Run the demo.js file

Intro to JSX

JSX stands for JavaScript XML. JSX is used to write the HTML-like code in JavaScript.

JSX is basically a syntax extension of JavaScript. React JSX helps us to write HTML in JavaScript and forms the basis of React Development. Using JSX is not compulsory but it is highly recommended for programming in React as it makes the development process easier as the code becomes easy to write and read.

Below is a simple example of JSX syntax used in a React component

JavaScript

```
import React from "react";
const HelloWorld = () => {
  return <h1>Hello, World!</h1>;
}
```

```
};
```

```
export default HelloWorld;
```

- The HelloWorld component uses JSX to render an h1 element.
- The <h1> tag is returned directly, showing how HTML-like syntax can be embedded in JavaScript.

1. Embedding JavaScript Expressions in JSX

JSX allows JavaScript expressions to be embedded using {}.

JavaScript

```
import React from "react";
```

```
const Greeting = ({ name }) => {
```

```
  return <h2>Welcome, {name ? name : "Guest"}!</h2>;
```

```
};
```

```
export default Greeting;
```

- You can embed JavaScript expressions in JSX using {}.
- The name prop is dynamically displayed. If name is absent, it defaults to "Guest".

2. JSX is an Expression

JSX can be assigned to variables, passed as arguments, or returned from functions.

JavaScript

```
const element = <h1>JSX is just an expression!</h1>;
```

```
const App = () => element;
```

JSX works like any other JavaScript expression.

3. Specifying Attributes with JSX

JSX uses attributes similar to HTML but sticks to JavaScript conventions.

JavaScript

```
const ProfilePicture = () => {
```

```
  return ;
```

```
};
```

Use camelCase for attribute names like className, htmlFor, and tabIndex.

4. Specifying Children with JSX

Elements can have children nested directly within them.
JavaScript

```
const Greeting = () => {  
  return (  
    <div>  
      <h1>Hello!</h1>  
      <p>Welcome to our website.</p>  
    </div>  
  );  
};
```

JSX supports nested elements as children, making component structures intuitive.

5. JSX Prevents Injection Attacks

JSX escapes values before rendering them, preventing injection attacks.

JavaScript

```
const userInput = "<img src=x onerror=alert(1)>";  
const SafeContent = () => <div>{userInput}</div>;
```

The above userInput is rendered as plain text, ensuring security.

6. JSX Represents Objects

Every JSX element is converted to a JavaScript object.

JavaScript

```
const element = <h1>Hello, JSX!</h1>;  
console.log(element);
```

The element above is an object with properties like type, props, and children.